

Active Inference ModelStream 007.2 ~ pymdp

Source: [YouTube](#) Playlist: ModelStreams Duration: 2:08:41 | Uploaded:

Unknown Transcript method: auto_caption

hello and welcome this is model stream number 007.2 it's the 18th of January 2023 we're back in our second session on Pi mdp welcome back Connor thanks for joining again and off to you for a presentation that you will weave together with some code examples and the notebook that we'll use is in the video description so thanks again Connor off to you great thank you Daniel it's great to be back here again I think I've been here maybe four or five times now so it's always a pleasure to be with you all um so yeah as Daniel said this is the second um model stream in a two-part Series where we'll be discussing implementing active inference with pi mdp so um just like there's some background that I'm assuming you either know from your own research into active inference and palm DPS in particular or because you've watched the first part of the series so um I'll just say that to start with like if you want background I would encourage you to go watch the first part where we talk about Pi mdp the motivation for the package uh what you can do with it in kind of sweeping terms and then today we're actually going to go in and use it to code up an active inference agent performing a task like a reinforcement learning style task um that's very similar to Classic active inference tests that you can find in the literature um yeah so as Daniel said I'm gonna step through these slides I have pauses for questions and stuff but we also have pauses for code um live coding or side by side coding so I'll take a pause from the slides and we'll go over to the um a collab notebook the link is in the description of the YouTube video and uh and we'll kind of code up this agent based on things that we discuss in the slides um it's not that I'll be actually writing code I'll just be running cell blocks um and yeah so if anyone who's viewing um wants to do it themselves you can open the collab notebook actually let's start by doing that because um so this should be the co-op notebook that you get brought to when you um click on the link and then and this is one copy you need to have a Google account to use this because Co-op is linked to your Google Drive of your Google account so if you're doing this um on your own this is a shared link but it's not um you can't edit it you can't write to it so what you have to do is go to file up here then save a copy and drive and then you'll get your own personal private copy of the notebook and I'll do that right now because I'll create a run copy where we can like

manipulate it all together in real time so I'll just call it run copy um and I'll start by just installing into this collab environment uh Pi mdp so collab is kind of like Jupiter notebooks it's an interactive Cloud hosted notebook for running different kinds of code but mainly python code um so while that's installing and I'll run that as well we can start with the slides Okay so um so I'll just start with a quick review from last time and again go back to the video if you want to see the full picture of what we discussed so the generative models the kind of brains of the agents that we'll be constructing are these partially observed Markov decision processes or Palm DPS um they have various parameters and it's kind of up to you how to parameterize them but under active inference we typically have these four main components there's fifth and sixth depending on how complicated of a model you want to create but these components kind of represents the agent's beliefs about the world that it's operating in so these are usually hand designed or at least generally structured on a case-by-case basis according to the behavioral task that you're trying to model so we have the A matrix or A array which encodes the agent's beliefs about how hidden states of the world that it's doing inference about relate to the observations that it gets you have the B array which encodes uh beliefs about transitions or beliefs about Dynamics so how do hidden States relate to each other over time um you have the C vector or C array which encodes agents prior preferences for seeing certain sorts of Sensations so this kind of plays the role of a quasi-reward function and it has the effect of biasing Agents action selection such that it's more likely to visit states of Affairs that it a priori expects to see I.E goal directed Behavior wants to achieve some goal and then we have the D Vector which is kind of a baseline prior about what is the state of the world um before I get any observations like what do I believe the hidden State distribution is and again I'm glossing over this because I assume that you know about things like categorical distributions um you know the we the all the details of those these different arrays are explained more in the first video and just again to show the graphical model we have the A array relating hidden States in red to observations in blue we have the B Matrix or B array that relate on hidden states to themselves over time so how does the world change and then we have policies π_i which are uh sequences or collections of actions that affect Transitions and thereby change the state of the world and then you have this critical quasi-seudo value function to active inference in Palm DPS active inference in general which is the expected free energy and the expected free energy is the thing you try to minimize as you optimize your beliefs about policies so the best policies are those that minimize expected free energy and that has a bunch of interesting terms that lend active inference all of its interesting curious uh information seeking behaviors and

then this prior over observations factors into the computation of that expected free energy so if you have a particular prior preference to see some observations that will affect which policies get more or less expected free energy and then you use that to actually choose a policy and then of course we have the D Vector which parameterizes the initial uh beliefs about his and states so today um we're going to actually dive in and write out our own a b c's and D's for particular um a generative model we don't have to worry about writing it out our own functions that do inference and or do planning all of that will be handled by the abstractions provided by Prime DP per tip particularly in the agent class so the Aging class takes the components of the generative model it gives you this very kind of Black Box API that you can just use to perform active inference um this is a very in the last video we talked about different levels of abstraction this is like the top level of abstraction in pi mdp where you're just passing a generative model to an agent and then kind of pressing go on the agent um having it interact with the task environment so typically this is another class like an environment class or a task class and then you just kind of string the two together in an action perception Loop and uh and you get a simulation of behavior under active inference and this is a a standard thing you'll see in like open AI gym or other reinforcement learning packages you'll have usually like an environment class and an agent class and they exchange actions and observations in order so we we designed the API of Pi mdp very much uh inspired by that um Okay so before we start writing down the generative model so that a b c and d um I want to spend a little bit of time exploring the idea of factorized State spaces and this is important because not only the model we're working today it's important to understand factorization in both observations and hidden States but for almost all the most interesting Palm DP models you're going to want to build it's going to be really crucial to factorize your system in some way not only because it becomes easier to reason about and work with when things are factorized but also it handles a lot of computational explosions that happen if you don't factorize your state space [Music] so it's not it's often useful to categorically separate observations into different modalities and hidden States into different factors so what this means is that when we get an observation we're actually getting a collection of different observations one coming from each distinct sensory modality or channel so similarly for hidden States at any given time the hidden states are described by a distinct set of different features or what we call Hidden State factors and so we refer to this as a factorized representation because the different modalities are independent of each other and the different hidden states are also um independent of each other their Dynamics are so they evolve in time without ex affecting the other hidden State factors so

that's formally written down as follows like the modalities are conditionally independent of each other given the hidden States that's what this factorization of the likelihood looks like so you can think of the different modalities as different sensory channels like visual Vision audition somatosensation and they provide their own distinct type of information which are then integrated together during inference and factorizing the hidden States is like the idea that any given time the world can be described by independent features like an object being described by both its location and space as well as its identity and factorizing the state's representations of the model in this way allows for a few different distinct advantages one is computational efficiency both in terms of memory and CPU it simplifies the structure of the generative model itself so it lends it a bit of interpretability and transparency so there's only certain sets of variables that represent the agent's beliefs about one feature of the environment for instance and then finally some would argue it has a degree of neuronal or biological plausibility and is consistent with the idea of factorized or what others would call Modular representations in the brain so this part of this group of neuronal populations deals with representing where something is in space and this group of neuronal populations deals with representing the Identity or what the thing is so that that's kind of the reason we do this factorization so one of the most classic examples of uh the the early examples we do in pi MVP and this can be found on the uh documentation and on the main page is this a grid World example where the agent is kind of navigating in a 2d grid World space uh then in the notebooks online we do like a two by three by three grid world so there's nine total locations and in a fully unfactorized or what we call an enumerated State space representation every location in Grid world has its own particular uh like Identity or state so we just arbitrarily can label them like location zero all the way to location eight and that that covers all nine levels of grid World um this is fine but it especially for a small state space like a three by three grid world but it leads to some kind of tax on our memory and our uh our interpretability a little bit because we have to remember exactly how our nine indices map to our uh the different places in Grid world so like if I'm in the middle of the grid I need to like have some kind of lookup table to say okay location four is uh like zero or one comma one I.E the middle location the grid in a factorized representation the grid could be represented as two orthogonal or independent hidden State factors one is something like the X displacement or the horizontal displacement which now just takes three values like which column of grid world are you in and then uh vertical displacement or a y position which is something like which row of grid world will dot am I in and so now each hidden state is not a single nine-dimensional Vector

but it's actually a combination or a pair of two hidden State factors so two three vectors instead of a single nine vector and then a given hidden State itself is picked out by a coincidence of these two hidden State factors so if the X position is one and the Y position is two then it means I'm in two are one comma two so like the middle bottom of the of the grid world and then once you do this so once we have a factorized hidden state representation it has implications for the way we construct the generative model so in a kind of simple generative model you would just have one B array that in the original uh grid World example just is like nine possible hidden States the nine possible locations of where I just was that maps to the nine possible subsequent hidden States so the nine possible locations I could be going to and then you'd have that nine by nine Matrix conditioned on the different actions so in the grid World demo the actions are typically local like move up move left move down move right um but once you factorize things you're going to actually now have a collection of B matrices and this is a consequence of the factorization across the transition models that we discussed a few slides ago so you're going to have one B Matrix or one set of B arrays for the X displacement so how does the agent move in the X Direction so these are what each of these X um these X displacement um B matrices look like conditioned on different actions so this is what it looks like from moving left this is what the B Matrix looks like for moving right and this is what it looks like for stay and then similarly you'll have another B Matrix that encodes the agent's abilities to move in the vertical direction or Y displacement and similarly they actually look identical um because the the world is is like rotation and translation invariant and you'll have a another smaller B array just for y movement this is a very simple example we're not going to actually do this in code because it's kind of trivial but it's just an example of how instead of having a location fully enumerated representation you can split that up into basically a y-axis and an x-axis so now we're gonna We're Not Gonna simulate an active inference agent but we're just gonna play around with these data structures to get you used to the idea of factorized representations and how that manifests in the code so now we'll go over to collab so if you've been running this on your own this should all be downloaded um so you've installed in infra actively dashed Pine DP that's the whole name of the package that's now in your environment and then you should also run this some basic Imports like numpy and Pi mdp which is now in the environment mm-hmm okay and so the basic variables that you'll see are things like the yes so there's a few variables that are almost always part of any pine DP workflow and this is very much mirrored on how it's done in SPM in the Dem toolbox so typically you'll have a list that specifies the dimensionalities or the number of different levels of each hidden State Factor

so for our three by three grid world nine possible States we have one X Factor of three levels and one y factor of three levels and then the length of that list tells us the total number of hidden State factors so in this case we have two hidden State factors and then similarly we have uh hidden states are factorized and then similarly you factorize your control States so for every hidden State Factor there's also going to be a control State factor and the fact that these are also three-dimensional each just basically means that the agent has three possible actions in the x-axis and three possible actions in the y-axis and then the number of control factors is also gonna be two in that case and so we have some nice utility functions that can quickly build a B array given the two things that are necessary uh the num States and num controls variables so you just do uh initialize empty B some states and number controls and then you'll have a B Matrix that has you know the correct shapes and stuff so three rows three columns and three possible actions for the X Direction and similarly for the Y Direction so this is just basic setting up your variables and often we'll have these solution cells that are hidden underneath each little quasi exercise so that's something you can do as well okay and so that's just and that's just initializing an empty array right so we don't actually know what's uh what those contingencies are that we saw in the slides but if we want to populate this thing so this is what each Factor specific B array looks like then we have to go into these matrices and actually fill out the rows columns and third order slices with numbers so for instance this Loop where we Loop over factors and we fill out the different entries of the B array what this will do is just encode those local actions like move left move right for The X Factor and then move up move down for the Y factor and so once you've run that code you basically filled out all the numbers in that that Big Empty B array that you started and then you can use things like the plot likelihood function to plot these likelihood distributions action conditioned likelihood distributions as matrices so this is move uh left for instance if we're in the x hidden State factor and if we change this to a one we've changed the action index then this is move right and then similarly if we change this this is move down if we're doing action one here um oh and just uh so darker colors whether it's grayscale or red or whatever almost unanimously in Prime DP the darker colors represent more probability so that's just for so instead of having to display numbers we'll just show darker numbers okay so now let's go back to the slides and talk about the A arrays so in in our unfactorized fully enumerated grid World example and again I'll refer you to the documentation of primevp if uh if you want to like play around with that example the hidden State factors which are just these nine kind of arbitrarily labeled locations map to an observation modality which is like

the agent's sensation of its own position kind of appropriate receptive GPS modality you could think of it as like they're sensing where they are at any given time and so um there are various ways you could uh encode that but let's just pretend that they have a very high fidelity GPS so they're getting precise GPS readouts of where they are at any given time so in in a fully enumerated world that a matrix that likelihood modality would look like this when we start factorizing things so splitting up the hidden State factors into an X Factor and a y factor that choice not only manifests in the B arrays but it also manifests in the structure of the areas and I think this is one of the most key things to understand because when it comes to coding these things up yourself understand how these arrays are structured is really important to getting things working like successfully so it's basically getting used to multi-dimensional indexing so the briefly uh in summary when you have multiple hidden State factors that manifests as extra lagging Dimensions or extra higher order dimensions in your um in your a array so it's not really an a matrix anymore it's an a tensor so that means if you have two hidden State factors the number of lagging Dimensions I.E excluding rows that's what I mean when I say lagging Dimensions will be two so you have columns and slices in addition to rows so this is really important to understand when either using SPM or pi mdp to do pom DP based active inference and another way of saying this is for each additional random variable that we're conditioning our observations on we have an additional lagging dimension in our array so this is the same Principle as with the B matrices because we condition the next state both on the past State and the past action that's why the B arrays are also tensors so we have two extra dimensions in the B array one for each conditioning variable past State and past action so now in the in the X and Y factorized grid world we have two lagging Dimensions where the first lagging Dimension the columns in blue index a particular X location and the second lagging Dimension the slices if you will index a particular Y location so now what I'm going to do is unwrap these slices of the multi-dimensional a matrix and so you can see that each slice of the a matrix so that each purple indexed third order slice corresponds to conditioning on a particular setting of Y so a particular conditional distribution over observations for the three settings of X for a fixed value of y so each slice is exactly this conditional distribution where we're looking at the conditional distributions over observations for each setting of X so those are the The Columns indexed by the blue numbers at the bottom but we're fixing to one value of y so that's the conditional distribution over observations 0 1 and 2 so being in the top row given that we're in the first the first row y equals zero for the three possible settings of X and then we move down to this next set of observations and uh the next slice of Y and it looks like

that and then finally like that so this is the case where our GPS is uh noiseless and High Fidelity right that's why we have like hyper Dark Cells and then everything else is white is because the agent believes that its observations are High Fidelity um signals of the actual hidden State namely the X comma Y location so then when we Pat these back together we see how this full a matrix or set of conditional distributions will be represented by a nine by three by three tensor where again each lagging Dimension exists to encode the conditional dependencies between that particular hidden State Factor indexed by the lagging Dimension and the observations for this modality so when I index into the ith column and the jth slice of this a matrix or a tensor I'm basically slicing out a particular conditional distribution over observations where I'm deciding what settings of the conditional distributions or the conditional very random variables I'm interested in okay so yeah that's uh that's a kind of critical concept and to just uh make it more um tangible and clear and I mean to be clear this isn't the only way to encode conditional dependencies in categorical distributions but it's just the way that was originally uh used in SPM to encode these dependencies and then we just borrowed that convention when building a primevp and it's pretty useful because it makes generating things like conditional expectations pretty straightforward um using like a linear algebra operations like higher order dot products and stuff like that so we have this kind of unwrapped representation of the array here and we can use this to uh to refer back to when we're building our array so um one thing I didn't mention that is also important is that in the same way we can factorize hidden States we can factorize modalities so we'll have we'll talk about that in a second but just for now let's assume we have one single GPS sensing modality of nine possible levels of where I am in Grid world so it's just a list of one number nine and then the number of modalities will trivially be one the length of that list and then we can use similarly this utils initialize empty a array which takes the number of observation modalities their dimensions and the number of hidden States and then it creates um the a array with the proper dimensions and everything so I've also created this combined list called a location dims which is a combination of the number of observation modality dimensions and the number of hidden State Factor Dimensions so that's as we said going to be 933 and so then we can populate our first the entry of our first modality in the a matrix which is trivially here the only modality with a matrix of zeros this is actually what's happening under the hood in this thing so we don't need to do it here but then if we just run that then we'll see that the shape of a0 is the same as a location divs um okay so that's just initializing it so we now have one a array that has one modality whose shape is nine by three by three and then we just go and fill it out

like we saw up here so I'm filling it out under the assumption of this noiseless high high quality GPS sensor so you can see when I'm indexing into the lagging Dimension here the the Y Dimension which is the third dimension of the tensor I'm saying for this particular value of y which is when Y is zero um we're in the first row of grid world then the conditional distribution over the three settings of the GPS given all the settings of X is just an identity Matrix so I'm just coding that that little chunk right there and then we can go through each of the slices iteratively and do that um so yeah one of the things I think is a disadvantage currently of both Prime DP and the SPM is I think we need better ways to make people not have to do these multi-dimensional indexing operations to encode a particular conditional independency structure and I thought of ways to do that but it's actually pretty hard to do that in a generic flexible way where people can just come to the table with their own semantics and then all this gets done for them so for now you still have to kind of do this by hand or write some algorithmic way to do it that's specific to your task but uh but yeah that's just something to mention is like it would it's not ideal that we have users doing this but I at the moment I can't really think of a better way to do it and then once we've encoded that we can look at our little conditional distributions given different settings of why you know and that's just one choice we could also condition on X and look at for all settings of Y x equals zero like you can slice this up however you want right um that's just a a visualization choice and these are our three collections of conditional distributions one for each setting of Y okay you know let's go back to the slides yes so one thing I didn't mention uh or I said I would mention is we we just had a a single observation modality but just as for hidden States we can factorize um and we don't have to fully enumerate our observation space so we could separate the grid GPS observations into an X and a Y GPS so now you're observing a pair of different observations at any given time One X observation and one y so just like we had a separate B array for each hidden State Factor similarly we now have a separate a array for each observation modality so we would populate one big a object array with these different modality specific A arrays so in this case if we had X observations and Y observations we would have two a sub arrays each one would have size three by three by three because three observations and then for the two possible states of the Hidden State factors so yeah that wraps up the the um slides I had on multi-factor multi modality factorized representations and how that affects A and B arrays so uh before we move on to the contextual multi-arm Bandit um let's open it up for questions or comments if anything was unclear and someone wants to contribute their own way of explaining something I'm happy to talk about that now awesome Jacob then Adam and Carl if

you'd like yeah uh thanks a lot for the great uh overview I had a question on uh initializing the B Matrix so in this case we uh initialized it with the simplest policy just all of the available actions the agent has is there would there be any particular use case in the grid World example where we would want to initialize it with a sequence of uh of policies and is that needed if we want to do planning of different temporal depth So when you say sequence of policies can you give me like an example like instead of in instead of um conditioning uh conditioning it on just move up viewed conditioning on move up and down fixed sequences of of actions right right that's interesting okay so that's so yeah typically when we are building these transition likelihoods this is more of a consequence of the fact that it's a palm a p-o-m-dp with an emphasis on the m so it's markovian so if you want to encode something like what is the predicted next hidden State given I did move up and move down in the last two time steps if I'm understanding correctly you're like saying can condition my B Matrix my next hidden State on a sequence of two possible uh past actions then we're breaking the Markov property of the Dynamics because we're saying the next thing says not just depends on what I did at the last time step in the end the state but it depends on also what I did two time steps ago so now you have a higher order temporal dependence like a semi-markov model or uh yeah really just not a markup model but a higher order model and that's not something that's currently supported in pi mdp the natural way you get around that is by building a hierarchical model um so at at each individual um level of the hierarchy you have a pom DP but then when you look at a single layer like the bottom layer of this hierarchical model it is a semi-markov model or non-markovian if you take into account what all the other layers are doing um but but that's not uh currently supported at the level of a single Markov model like you can't build a b Matrix that's dependent on more than what happened at the last time step it always has to you yeah you can't have like an extra lagging Dimension that encodes also what were they doing two time steps ago that so that's one interpretation of when you say policy like up down I think of two actions uh that happen in in locks in sequence um but maybe another thing and correct me if I'm uh reading too deeply into your question another thing would be can I have actions in a different control State Factor affect the Dynamics of the of a particular hidden State Factor so say I had one hidden State factor which is move up uh and another uh I said one factor which is y displacement another Factor that's X displacement can I have the Dynamics of the Y Factor not just depend on my y action but also on my X action um so that's the idea of basically breaking this uh conditional sorry this Independence property where is it here I don't know let me find it here so if you you allowed the lower right shows that the

dynamic the state of one factor over time only depends on the state of that factor at the previous time step but if you said I could also make this conditionally dependent on not just factor F but say Factor u or Q at T minus one this is something that's also not currently supported but I think is an interesting idea which is you would actually have interactions between maybe pairs of hidden State factors in in Hidden State space this actually can be accommodated under uh Pine DP like like mathematically I haven't built in the functionality to do it but I think there's nothing that stops you in principle from doing this so this would be as if I can have actions from different control factors all influencing the next state of one hidden State Factor u and that is something that will probably make the message passing a little bit more not maybe not more complicated but maybe there's convergence properties wouldn't be as guaranteed I don't know what it looks because you're introducing weird Loops in your graph but um in the factor graph that represents the generative model but uh that's something in theory could be uh entertained but I I don't know if that's actually what you had in mind maybe you had more in mind of breaking the Markov property when you say yeah it was um thank you for the answer it was more of a clarification also on the mathematical notation uh because uh π uh it's interpreted as policy which I guess can be both individual actions and sequences of actions um yeah so just wanted to clarify that yeah that's a good point so when I said π up here I meant I meant yeah so in in if Paul if π was a sequence of actions that any any given time the B Matrix that you're indexing out is conditioned on the action that's entailed by the policy at that particular time but the B Matrix will never be entailed on like what's going on in the full policy but just by okay at time t i take out this slice of the B Matrix because at time T this is the action entailed by this policy of sequence length H or something but uh yeah that's a good point so in the in in a trivial case policy length is one and there's just one action that's uh the policy is just an action that comes down to slice out the b Matrix thanks Jacob Adam anything you like to add go for it yeah a couple follow-ups uh thank you for this very clear presentation Connor so first um just to follow on what Yakov just asked it seems like another potential way if you wanted to do this sort of two-time Step uh model would be to actually condenser or kind of permute those two time steps into a single uh variable the same way that you you know could condense the row and column into a single nine Dimension you know nine nine row vector or you can break it down into two could you similarly break down the last two moves and sort of combine them into a permutation of all possibilities of the last two moves and have those be the hidden States yeah uh you could do that as well I don't I haven't seen that actually done in practice the only thing is you'd have to

actually have to somehow change the there you have to change the the scheduling I guess of the message passing in the action to accommodate the fact that before I determine what my last hidden state was I need to like wait two time steps so I can get not only the proximal action I did but also have this memory of the action two time steps ago um yeah I was imagining that you'd have some special special case for the first move where it's like undefined and then the last move then after that you'd have the last two moves a separate Matrix for those or something yeah like a moving window kind of so it's always yeah remembering the last yeah I I could see that working um I think like the active like kind of the canonical active inference answer to that is instead of having to kind of manufacture this special way of represent of basically baking memory into a markovian system the classical way to handle that is just to build a hierarchical model because the hierarchical model handles that memory um easily by just having like nested markovian processes where if you if you were to look at a single process it looks like it has memory but that's just because it's priors for its own markovian process are evolving as a function of another Palm DP that's operating at a slower time scale um so like I I could imagine doing that exact exact thing that you're talking about if you had a low-level Palm DP that was moving twice as fast as the top level Palm DP and so basically with the top level Palm DP is doing is setting an initial prior over one action that then allows the slow level Palm DP to condition whatever it does next on the behave on like a belief that whatever I do next I know that I just took um this action two time steps ago but you don't actually have to hand craft that it's just by building a one Palm DP like within another Palm DP um but I don't see that there's anything in principle going against what you're describing as well it's just yeah I like your approach a lot better yeah this is this is um I mean essentially what I learned from from working with Carl was that like any time you're trying to imbue a system with more complex non-markovian Memory full Dynamics or higher order dependencies that's when you it's the natural move is to go to a hierarchical model um because it kind of handles it all naturally so my next question is um about the uh what you were sort of tweaking in that formula just a couple minutes ago where you were suggesting the possibility that there could be feedback loops you know between the hidden States in theory but but as currently implemented if I understood correctly that's not the case the state of us of one of the Hidden variables or so in the state of one of the Hidden States depends only on the previous dates of that the previous state of that hidden State as currently yeah but I want to make yeah a hidden Factor what I want to make sure I I understand is that is that also something about the markovian Assumption or is that just sort of an implementation or design decision completely

separate from the markup yeah that's that's a good question that's separate from the Markov um that's separate from the Markovian Assumption so that's something having to do with kind of just how the graph that represents the generative model which you could think of as like a Bayesian graph like a bunch of nodes that uh affects a layer of observations so you say a bunch of hidden State nodes that all uh all have all the observation nodes conditionally depend on the state of all the hidden State Factor nodes by constructing it this way you basically have a graph that it's very easy to do inference on that has like stationary fixed points so the inference does not depend on like initial conditions of the posteriors and things like that so um this is like a kind of a bar the way the generic model is set up right now is like a bipartite graph that resembles a classic um uh model for machine learning called like a restricted boltzmann machine so this is where all the hidden States can all affect one uh observation modality but there's no interactions between the observation layers so there's no lines going between observations within one modality and there's also no lines going between hitting State factors so that's that's kind of what we're what one of the advantages of this assumption is is when you have the hidden State factors be only dependent on their own past State and their own past action within that control Factor then you're not introducing these Loops in the graph that make inference a little bit harder so you'll see this structure in when people are using like restricted Boltzmann machines or other or even deep neural networks it's very hard to get a fixed Point solutions for like the posteriors when you're when you have interactions between nodes in a single layer so it's not quite the same thing as as um it's yeah it's just not the same thing as a the Markov property has more having to do with generating an acyclic graph so you can do a fast efficient and fixed Point inference on it I mean that that's my general answer to that because if you look actually draw out the factor graph for these things it looks like that but there might be another reason this is done that I'm actually not aware of um maybe that's something that Carl uh would know but or or someone else if you want to add anything Carl otherwise and then after this we'll head into the script go for it Carl thank you um yeah I think everything that needs to be said has already been said very very clearly and very usefully uh but yes Connor's absolutely right um so I look at this um in terms of the overall architecture of the graphical model you bring to the table and you know it can only have a number of different attributes you know how deep is it how many levels does it have um we're not in this presentation talking about uh deep uh um mdp or generative models based upon um hierarchically composed uh Markov decision uh processes uh but we you know generally you would think of any one level in the context of the level above and the level below and that

lends this attribute of depth to Any Given generative model but probably more important certainly from the point of view of the current discussion is the Barrette and the barrette's basically the number of uh independent actors in physics this is simply known as the field approximation it's you know it's just the factorization of a joint distribution in this instance what we're aiming for is an approximate posterior distribution but it's a factorization into conditionally independent factors that incidentally also induces certain Markov blankets and resolves a lot of the message passing and uh means sort of variational iterations are more robust but um from the point of view of your why do you do that well you do that because to maximize the marginal likelihood of the evidence for your generative model you have to carve the nature out there at its joints in the right kind of way so if the world you're trying to navigate or exchange with or explain um does have the this factorization is conditional independencies it is carved in this way then your model has to comply in order to have the maximum evidence um so when learning that particular factorization or that structure you would normally apply a process of basic model selection or structure loan to get the number of factors right so and that is getting the right kind of mean approximation apt for this world and and these data and the right factorization will simply minimize variational free energy and therefore minimize the computational complexity and also the uh the computational complexity expressed in terms of thermodynamics for example so it's a really important sort of thing and there's a couple of the little um endorsement comments um you know conditioning any one um state with any one factor on States and other factors destroys that that particular factorization so you don't have to worry about introducing loops and things all you all you're doing is saying that particular factorization has gone away you've coarse grained and you now have to lump together all the elements of one factor and the other factor into one bigger factor and now you've got a couple more coarse grain factorization and more mathematically complex model because you haven't done as many carvings uh in you know tourism you know to Leverage The conditional independencies that you um you're trying to model and the first question about the policy is that that was an interesting one I suspect it may be an artifact of um the way that we condition everything on I without sort of forwarding people I is actually quite a complicated variable um first of all it's basically you're an index or a a name for quite a complicated combination of things so not only is it a sequence of um actions um upon which you condition each individual transition at any point in time over time so it's a sequence of actions so an answer to Jacob's question um you know the whole point of the expeditary energy is exploring lots and lots of different sequences as you run out into the future but also

um induced by the factorization as Conor was saying um the that pie actually has to now entail a action for every factor of a fixed combination so if you change the the combination of actions for two factors you now have a different policy even at one time step even for one step ahead so the notion of a policy as a combination of actions uh is still in play when you have a factorized mean field approximation of the Hidden States so if you know it's a subtle Point usually would resolve it by actually explicitly writing down usually the using the variable U the thing that you're conditioning the state transitions on and then you um comes from a family of combinations of views over factors and over time um so you can have deep and shallow policies but even you've got very shallow policies one step ahead and one look ahead apologize there's still a subset of combinations of actions you know so it could be that you can either only move up or or down sorry up or to the left but you can't move diagonally for example yeah and so and that will be placed um in terms of priors over Π namely combinations of of actions oh so one final just for for Adam's benefit that notion of um putting semi-markovian Dynamics and I'm getting kind of absolutely right that the other the way that um as a physicist the way you do that is by appealing to the depth of the hierarchical model which always entails time and that's that's an absolutely crucial observation there's no point in adding depth to a generative model unless you are mindful it is introducing a separation of temporal time scales and I think you know that's a profound this sort of a structural insight into the nature of genital models and has all sorts of implications um but then it you know just speaking to this notion of sort of having combinations of little histories or little caches of Legacy State um as if you like a superstead um strictly speaking that is exactly what is done effectively in continuous text-based models with common filters so you you actually have now two kinds of random variables the position and the velocity so you you've now got a sort of dual representation and if you generalize that you get a generalized points of motion so you know and technically um when you're doing variational message passing you now use something called a chef face free energy as opposed to which is a variant of variational free energy um however when you move to discrete States there's generalized coordinates now actually are replaced just by having um states that unfold in time over the um under the public transition Matrix so that you you shouldn't in principle ever need to do that but you can now regard if you like the little local history the little little packet or trajectory or orbit the path that you're taking at this point at this Temple scale um as sufficiently discretized and cause brain with the different probability transition matrices and then at the slower time scale then it's a succession of initial States that's normally how we would think about that but it's a it's an

interesting game to think about how what kinds of structures in these dispute States based models Echo the equivalent movements one has to make in continuous State space modeling and uh and thinking here particularly generalized filtering and Bayesian smoothing and Bayesian filtering and the line yeah thank you that's a really it's really nice to have uh those afterthought like because that clarifies I think and justifies a lot of the preceding slides or what what Carl just said yeah that's that's a good I I so you can kind of map the depth of the hierarchy think of the depth of the hierarchy as a discretization of what is smoothly handled with generalized filtering and generalized coordinates of motion um so States evolving at this Pawn DP are kind of like a quasi-velocity variable uh because they are evolving slower than the position variable which is like the low level quick faster clock uh Palm DP one one just tiny footnote there is it relates to the continuous and differentiable free energy landscape and the discretization that is required on Modern digital Hardware around sampling and the discretization problem on a continuous but hidden function and the approaches that we have to take to discretize the continuous nature of time especially absolutely so I think we should proceed to the um the multi-on Bandit um okay so now having discussed these kind of factorized representations we're in a position to build our multi-factor generative model for what I'm calling a contextual two-armed Bandit or contextual multi-arm Bandit this is coming from the um you know reinforcement learning literature This this term contextual but uh it's based directly on a kind of gambling task that was introduced in Ryan Smith Christopher white Carl's friston's step-by-step tutorial on active inference so you basically have a multi-arm bandit two-armed Bandit but you have to first find a clue or uh forage information to figure out which of the two slot machines or arms of the Bandit is more favorable and this is echoed as well in the original teammates example where you have a rat that has to choose between two arms one is more rewarding than the other but you don't know which one is rewarding so you have to visit a informative Q State first that tells you the left one is rewarding right now or the right one is rewarding so this I'm just kind of gathering all these things into the term contextual Bandit because that's kind of the analogous uh term for this problem that you'll find in like reinforcement learning world um okay so in the multi-arm Bandit or two on Bandit task an agent has to play choose to play one of two slot machines and they can only play one at a time so one of the slot machines gives more reward than the other but the agent doesn't know which is the better one so at any given time the agent can choose to play the left or the right machine that's what we'll call them or can choose to ask for a hint and if it chooses to ask for a hint instead of playing the machine it receives information about

which of the two slot machines is better so this sets up this already sets up the basic dilemma of tasks like this the so-called explore exploit dilemma so you first Forge for information that will ultimately lead to getting more reward or do you gamble immediately in order to get rewards sooner but at the risk of not having enough information to know which uh slot machine I should actually play so to encode this task structure into a generative model for a palm DP we equip the agent with three observation modalities first we have this hint modality which is the sensory channel the agent uses to perceive the hint so just what is the state of the hint then there's a reward modality which tells the agent whether it won or lost at the selected machine and then finally there's a choice modality which is just the agent's proprioceptive observation of its own choice this is equivalent to the GPS modality that we were working with in Grid world and in terms of hidden State factors we first have a context factor which is a binary variable encoding Which slot machine is the more rewarding one either the left machine is better or the right machine is better so those are the names of the two context variables you could just arbitrary enable in my context a context b or something like that and then we have a choice hidden State factor which is simply encoding the choice state that the agent currently occupies um and finally as Carl was just explaining for each hidden State Factor you also have a control Factor so we'll have a context control factor which as we'll see is Trivial since the agent can't control the context in in this um simulation and a choice control factor which is the control factor that allows the agent to model its own actions or choices so even for a shallow policy Horizon as Carl was explaining a policy will still be comprised of two actions so it'll be there will be a context action and a choice action but as we'll see the context action will always be to do nothing or basically to not be able to interfere in the Dynamics of the context Wednesday factors so basically action selection comes down to just choosing the state of the choice control Factor on that you Choice variable so now we're going to look at the different levels of the Hidden State factors so as we said the context one is a binary um or you could think of it as like a Bernoulli variable so it's either left bed or right better it's a distribution over these two states of the context and this is just the unchanging state of the world for a particular trial either left machine has a better payoff probability or the right one does and then the choice hidden State Factor has four levels which correspond to the four possible Choice states that the agent can be in so this can either be the neutral starting location on the hint state which is the location they they occupy when they're acquiring the hint um and or they can be playing the left machine or they can be playing the right machine and so now for the control State the control State factor for the context only has one level you can call it

do nothing or it's just a trivial one-dimensional variable and the agent will always take this action with 100 probability so in other words the agent um has no control over the state of the context and then the choice control state has four levels corresponding to the decision to change um the S Choice the hidden State factor for the choice state in one of four different ways either it can move to the start location it can move to the hint location it can play the left machine or it can play the right machine so what choice State the agent is in is kind of obviously under the agent's control which is encoded by the fact that this u-choice variable has four levels so optimizing a posterior distribution over this Choice uh control State Factor that's that's what action selection and planning boils down to is basically choosing what level of you Choice I'm we're inferring what uh the distribution over you choices and then sampling or taking the ARG Max of that distribution to actually make an action okay so now let's review the observation modalities and their different levels the first modality is the hint modality which can either give the outcomes of null like I'm not at the hint state so I'm getting no sensory information from that channel a left is better hint hint left or right is better hint and they're just named that because you will see in the a matrix how seeing that observation relates to seeing the um two to the the state of the context um we have the null observation because as I said if the agent isn't in the choice state of getting a Hint it still needs to receive some information from that modality so that's why we often have in both the SPM and the pine BP implementations we have this null observation that just encodes an observation that has uh has no information about hidden States within uh some particular Factor and then we have a reward modality which similarly has a null observation for when the agent isn't playing either slot machine and then two possible reward observations either a loss or a reward but this is a a choice that can um depends on what you're trying to do you could also make a reward have like 10 different reward levels that have different like magnitudes of rewards it's just a choice to call these like loss and reward one the point being that one is preferred relative to the other and then finally we have the choice modality which just allows the agent to unambiguously infer what choice State it's in so it just allows it to infer what it's doing and this is important because remember in active inference everything has to be inferred everything is about is in the game of minimizing free energy and um coming up with uh posteriors approximate beliefs about the state of the world including your own choice state so that's why we often equip agents with a proprioceptive sense of where where I am like a GPS yeah okay and and so that that's the three observation manage we've done the hidden States and the controls so now we're in a position to start building the a arrays for this agent's generative model so for each

modality specific a array we're going to have as many rows as there are levels of that modality and then two columns and four slices and so why is it two columns four slices because each column corresponds to a setting of the context hit and state factor and each third order slice corresponds to a setting of The Choice State hidden State factor and these lagging dimensions of two columns four slices will be the same for every single a matrix not not the values in them but that shape so the only thing that will change across modalities the a uh modality specific a arrays is the number of rows which is the number of observation levels so for the hint modality each slice of this a matrix tells us the mapping between the two possible context States and the observations in the hint modality for a fixed Choice state that the agent could be in so for instance if the agent is in the start Choice State what we're seeing now regardless of whether the left or the right mission is better the agent will always get the null observation which has zero information about the state of the context as you can see because given an observation of the null observation in the hint modality you have no idea whether the left is better or the right is better however if the agent is in the visit a hint state or acquire a hint State then the mapping between the contact State and the two hint observations will be informative and why I I say informative I just mean that the columns of this Matrix will uh be independent from each other and they will be low entropy so another way that's saying that is that there'll be low entropy in the observation conditioned rows of the Matrix so uh briefly what we're seeing here where grayscale darker colors means more if the slot less machine is better that's the context then the agent is more likely to see the hint left observation and if the right machine is better they're more likely to see the the hint right observation so this slice of the a matrix is what gives those observation levels their their meaning the the very region they're called Hit left and hit right is just because of the the kind of diagonal structure in this slice of the Matrix and then finally for the other states um if I'm playing the left or the right machine I'm never getting any informative observations from this modality um okay and then we move on to the reward modality so again two columns and four slices like for all the arrays but now we're looking at the mapping between the two context and the reward observation levels so no loss reward for each possible Choice State the agent can be in so as we can see if the agent is in starting location or getting the hint they're not getting uh any reward observation they get the trivial meaningless null observation however if the agent is playing the left or the right machine though there's a probabilistic mapping between the context the left machine or the right machine being better and and uh the expected rewards so if the context is left is better then the agent is more likely to see the reward if and only if they're playing the left machine uh

likewise if they're playing the right machine this Matrix uh is the same so if you're paying right then you're more likely to see reward so this is this sub Matrix within this big a tensor is what determines the payoff structure of the task um or the agent's beliefs about that payoff structure I should say because this is the area it's not the actual rules of the game so now we're finally up to the choice modality which again is just their observation of Their Own Choice state which you can see only depends on the state of the choice that they're making it doesn't depend on the context so it just means that they'll always unambiguously infer uh whether they're playing the right machine or they're in the start state or whatnot so this is just an example of going through those slices and really seeing how that looks so now we can code this up in collab and again we won't be doing it by hand we'll just be um we'll just be running some predefined code so one useful thing to do often with control factors and observation modalities is get some semantic labels like create a list of strings for instance so the context names will have these two states left better right better choice names will have these four possible States and then we can automatically create these lists of Dimensions that are needed to create your A and B matrices just by looking at things like the length of this list of strings or the length of this list of choice names so if you run this then we'll have our num States variable which will be 2 4 2 possible contexts and four possible Choice States and then you'll have num OBS which will be yeah three possible hints three possible rewards and four proprioceptive location or Choice State observations and then we can use this function to initialize our empty a matrix and in the next few chunks of code basically just fill out these a matrices like we saw in the slides um so one thing we'll do is we will have two parameters and this is again arbitrary you can choose to do this however you want we'll parameterize the a matrix with a probability of hint or a hint accuracy which will effectively just fill out that slice of the hint modality a array that corresponds to how accurately the hint um signal uh indicates the context or or lends evidence to the context so if the hint is 100 is 1.0 P hint is 1.0 it means whenever they visit the hint State they immediately know okay the left arm is better um but this doesn't have to be 100 right the hint itself could have some noise or uncertainty associated with it so we'll parametrize that just with this um so this is the probability of the two hint types or the two game States and you see the null has zero probability and then hint left has more probably when the context is left indicated by the column and then hint right has more probably when the context is right and then we'll just go through and fill out this Choice um stuff oh you the reward modality will have a similar thing a p reward so this will determine the payoff structure of both banded arms so if P reward is high it means if the context is left is better and you're

playing the left slot machine you'll have an 80 chance of getting the reward and this thing itself can be not changed to determine basically how how rewarding the bandits are so this is the payoff structure uh if you're playing the left arm the payoff structure for the two contexts and there's an inverseness that I didn't discuss so if I'm playing the left arm but the right arm is better then I'm also more likely symmetrically to get punishment or negative reward with the same probability that I would get positive reward if I was playing the right on so just that that's also a choice but just to make this a single parameter that's how we we did it here and then finally the choice observation model is just filling out that GPS uh High Fidelity GPS sensor and that's pretty um you know if you slice it along the any context hidden State Factor since it doesn't depend in context this could be a zero or a one then we'll get an identity mapping which just allows them to infer their choice state and then we wrap this all into one function so we just have two parameters P_{hint} P_{reward} and then we can see once we start doing simulations how manipulating those things messes with their behavior so now we'll move on to the B arrays um oh yeah so let me go here and talk through some slides this will be quick the bras are pretty simple so we're gonna mess with this later but we're gonna start by saying that the context hidden State factor is stationary over time so they can't change that end State factor and even in the absence of their ability to intervene it it uh it stays in the same state over time so we parametrize this with like a p change probability that just fills out the diagonal of this um kind of trivial transition Matrix with 100 probability but later on we can mess with that parameter and decrease it so they believe that the context can actually change with some stochasticity over time um but for now we just assume they don't believe the context changes and uh and then this is their ability to so unlike grid world where you have local actions they can only move up down left right here they can move from any other state to any other state so this is like a signature thing if you want a controllable um hidden State space so the agent can go anywhere from any other place that will manifest as these dark bands in your in your B Matrix or bands of 100 probability so this is another way of saying that the transition graph is fully connected that you can get anywhere from anywhere um but that's also not sure like we could have made it such that they once they leave the start State they can't return there so that would correspond to having um the absence of one of these uh uh like basically the start state is a source but it can't also be a a sink or if there's no uh in arrows going to it so that would affect the structure of okay and then we can quickly code that up um so b_0 is the context in state which as we said is just going to be stationarity and then the choice given state is it's fully controllable and then we will parameterize this all as a create B function which will for now

have this P change thing so we can make the agent's belief about the stationarity of the context um different like if the context is likely to change but for now by setting it to zero it just means that they believe the context is fixed for a given uh trial or set of time points and then finally very basic stuff this is just how we establish the semantics of the reward modality so the reward modality is as we said the null the loss and the reward and we can encode the C Vector which is kind of like the reward um the prior preferences or prior over directly in terms of the log relative log probabilities so we can encode punishment by having the prior expectation about loss be much lower in relative log in in Nats natural log units much lower than the reward observation within that modality and then the other ones are just going to be uniform distribution so you can leave them empty and this is just what allows them to want to see reward as opposed to loss or punishment and uh yeah so just as I think we discussed this last time um Daphne had a question about that so why do we encode things in relative log probabilities rather than Shear probabilities is just more analogous to the reward construct from reinforcement learning that's one benefit so and they're unbounded you don't have to they don't have to be bounded between zero and one so if you just encode them that way it becomes easier to kind of make one thing x times more rewarding or punish or aversive than another thing and that directly relates to how kind of reward is calculated in our expected utility of the expected free energy is because it's always kind of an expected it's kind of like an entropy or a cross entropy so it's always um it's you its units are in in natural log space okay so yeah um and then the D Vector again they can have prior beliefs about which context is better so that will also be a collection of vectors one over the context factor and one over the um the initial state of where they are they are which we can make it start but it doesn't really matter because they have precise observations and then that's very easy to do in Prime DP we'll just create some functions that parametrize how rewarding the reward observation is and how punishing the punishment observation is and then we can plot those and you know this could be 10 and then the reward is much more is much better than the punishment or the punishment could be very bad in which case the punishment is much much lower than both the null and the the reward and here I'm showing the belief the the prior directly in terms of probabilities so when you convert the log probabilities to probabilities these things will become bounded between zero and one okay and then the D Vector assembly will just create a quick function that parameterizes how much they believe the left is better is the left is better context is true at the beginning of the simulation and if you just set that to 0.5 it means they have flat unbiased uninformed prior beliefs about the um yes okay all right so now I think we are good to actually

do this after all the build up um so like 95 of the work is in encoding the generative model and this is something Carl will tell you as well SPM like most of the work is not actually running active inference maybe computation wise it is but in terms of the time and the intellectual energy spent it all comes down to the generative model that's where all the information is the rest is just optimization basically um so we've written down our a b c's and D's then we just basically plug them into this agent API from PI mdp and then we create an environment class which could be as something as uh ad hoc as I'm getting a list of observations from some like API that's talking to the internet or like from a robot sensors or you can actually write down an environment class that generates uh observations like like another agent for instance um and then you plug the observations from the environment into the in first States function you do infer policies and then you sample an action according to the expected free energy stuff that we talked about last time now we're actually going to do this so we have our nice Nifty functions for creating a b c and d we can just run those with desired levels of P hint and P reward and then create our agent and um this is something we discussed last time but an important distinction is the generative model versus the generative process or the environment so the things we're putting in a and b right now and and C are just the agent's beliefs about for instance the hint accuracy or the payoff Matrix of reward that can be arbitrarily different from the actual structure of the environment and we'll see how later we'll see how you can adjust the agent can actually learn the correct reward statistics through process of learning so even if they start out with the wrong generative model they can adapt their generative model online so that they um they match the generative process better okay and then we'll Define this quick class the two Arm Bandit which will be the generative process the real world for our agent we're not going to step through the code but um the link is available so if anyone wants to look at this this is basically just encoding the rules of the game so when the agent is in the left arm they'll get reward or lost depending on the P reward and P hint values and as we said these values can be different than the agent's beliefs about them which will be in their a matrix so this is the generative process the actual reward and then the generative model are the parameters you give to the a Constructor and then we'll just have a function here that runs the active inference Loop it looks a little bit more complicated than those three lines I showed on the slide but that's just because I'm storing things like the history of their choices the history of their beliefs um I'm having optional plotting of beliefs things like that and plotting of actions but this function basically that the key Things Are you get an observation from the you start with an observation which it will be from the generative process you do hidden State inference

using that observation to get your posterior over and this you can return if you want to plot it but you don't have to and then you'll do inference about policy so this is optimization of a posterior over policies which then gets translated into a marginal posterior Over Control States or actions that you sample from and you sample from them if this next line with sample action so the main three lines of any active inference process are inference about States inference about policies and then action selection which can be either deterministic or stochastic depending on your application and then the rest is just basically code that converts observations from the generative process into like the terms that the agent can understand and vice versa and then we have this helper function that plots the history of choices and beliefs in the true context over uh over time so yeah I just Waze through that quickly because it's not very important but we can if anyone has questions we can go back and dissect that code a little more so now all we have to do is to find the oh yeah you have your hand up you'll ask something uh yeah um maybe um maybe this is better better suited for um for the end but um in Prime DP there's also modules that are adjacent to the Asian class uh so I was just wondering how you um how we could make use of the different modules in the action perception Loop um because I haven't I haven't seen many examples of uh of using for instance the inference module or the learning module and how that would augment the Run active information yeah that's a great question um so I have another demo that we can maybe do another time that's uh building a hierarchical model in pi mdp so building a two layer model it's a visual foraging hierarchical model based on a paper that we did back in 2018 2019 and um and there you you're in this inner loop over time you're doing you're making use of these sub-module functions so say you wanted to do some special kind of inference that's not just the standard in first you could actually say Okay I want to do this particular weird thing that's specific to this inference process and you would just add lines here um like I want to take a function from the inference module and run like I don't know a different kind of message passing on it um like we're on to MMP or something inference dot or that there's an Algos module too that has different message passing algorithms and then you could optimize your hidden State beliefs using that message passing algorithm and then set them uh equal to the the like special thing you did just just with that oops so yeah there's no examples here but um I haven't done that here but it's definitely something that can be done or yeah there's another example I did like simulating an active inference equivalent of an Evidence accumulation drift diffusion style task where the policy selection was done in a particular way where you're only using certain components of the expected free and uh and yeah so so

basically there's other demos where you can like zoom in and not use the agent class and use sub modules but I haven't written those in like a publicly accessible way yet but I can the hierarchical one I can just easily make public I just haven't had time to put it on the docs yet but yeah that's a good question because yeah you're right most of the stuff right now is just the highest level implementation with the agent and now uh that to just run the active inference process we just I've made a distinction between generative process parameters like how accurate the hint is how uh how the payoff structure is and then generative model parameters which are used when creating the a matrix so the P hint end is the true hint accuracy P reward end is the true payoff and then just choose some time Horizon and run the active inference loop with that time Horizon and then plot the history of beliefs okay so here's an example um the top plot shows the agent's Behavior so white squares indicate what it was doing at any given time so time is on the x-axis so you can see it start in the start State then it gathered information at the hint and the reason this wasn't immediate is because it's beliefs about accuracy were only 0.7 so it needs to actually do some evidence accumulation before it's sure what the state is then it played right but as soon as it played right because that's what the hint was telling it as soon as it played right it probably had a a loss a loss observation which then messed up its inference and made it go back to the hint to accumulate evidence again before being okay the hint is really suggesting that I should go right so then it goes back revisits the right arm and then over time you see here the Red Dot is the actual true context and the grayscale indicates the agent's beliefs about the hidden state so here um the hidden State beliefs are now converging to believing that the right arm is better and I know I said at the beginning that all probability would be darker means higher probability but it's actually the opposite here so sorry that's confusing um so yeah right better is higher and their beliefs get more and more confident as they play so this is you know this this is the exact same thing you'll see with the teammates right they first forage for information that's driven by this epistemic value component of the expected free energy as their posterior over hidden States becomes more precise epistemic value goes down and then um the expected utility component goes up which is bolstered by the fact that they now have confident beliefs about the context and then that drives them to forge information um and then so like that kind of exercise that I would have people do if this was like in a classroom context is start messing with the Bandit probabilities and also mess with the agent's sensitivity to punishment so up here we made them have plus two relative Nets in their reward function and minus four for punishment but if you made this even lower let's say it made it negative six what would happen so now because

they're more risk-averse the um the risk of getting a loss observation if I was to play immediately becomes higher so now the agent basically requires more confidence before it's willing to actually play the left arm because the risk of getting it wrong and getting a loss is too great because their reward function is now shaped more risk adversely and then if I like decrease this even down to negative eight you can see that they never even go for play because even once they're very certain that the left arm is better it's still too risky given their beliefs about the payoff structure because there are beliefs about the payoff structure are such that they believe that there's still a 20 chance that they'll get lost but if we change this and said what if their P reward was 1.0 or say 0.99 then they're willing even despite the great risk of being punished because the the expected reward given you know the context is high enough they will still end up um risking it and they acquire the hint for some time and then they start risking it but they're the observations will because peer reward is so high their their beliefs about the payoff structure their beliefs will kind of oscillate as the observations actually change because the true observations are generated much more stochastically because this is the peer reward of the Bandit so you'll you can mess around with this and and like I would encourage if anyone listening now or afterwards um wants to mess with these parameters I would encourage you to just go crazy and mess with parameters and see what different kinds of behaviors you can get um yeah okay so that pretty much wraps up the main brunt of the tutorial and let's pause there for questions because I have additional material about learning parameters that we discussed last time would be good to to um get into because that's something that's really under documented right now with primevp so we can move into that but I think we should first break for some questions awesome thank you Jacob if you want to ask or Carl otherwise I'm happy to hear about learning cool let's do learning and then we'll have any closing questions at the end perfect I'll have some quick slides so learning under active inferences cast as updating beliefs about the generative model parameters itself so inference is one thing inference is saying given my generative model so my beliefs about the the way the world Works what is the best explanation in the sense of posterior over hidden States and policies that explains my actual data and you get that by minimizing variational free energy learning is much um becomes much more complex but also more interesting in the sense that if the agent's generated model itself is wrong they can change the generative model to also optimize variational free energy so learning and inference can kind of cooperate to or sometimes interfere with each other as we'll see to to minimize variational free energy so the way we have to start by doing this is now treating the parameters themselves so the parameters

of the $\mathbf{a}$ array which are categorical likelihood parameters $\mathbf{B}$ array $\mathbf{d}$ -array $\mathbf{C}$ array those themselves now become random variables over which we can have priors and of course variational posteriors um so we can go back to our original $\mathbf{a}$ or Palm DP representation so now what we see is in this third column next to $\mathbf{a}$ $\mathbf{b}$ $\mathbf{c}$ and $\mathbf{d}$ we have parameters that are priors over the categorical um parameters of the $\mathbf{a}$ $\mathbf{b}$ $\mathbf{c}$ and $\mathbf{d}$ so a natural um prior for these sorts of categorical variables is called the dear Schley distribution which is a um it's basically a conjugate prior for a categorical likelihood distribution that's why it's very nice and also the values of its parameters have a very nice intuitive um feeling and interpretability to them so now the agents will have priors which are updated by this $\mathbf{P}$ of Φ at the top um and Φ is just a collection of all these dirichlet hyper parameters they're also called or prior parameters and these are parameters or priors over random variables that themselves are the likelihood distributions and priors of the generative model so we're just going to do a few instances of learning today I don't know if we'll have time to do $\mathbf{b}$ or $\mathbf{d}$ learning we'll start with a matrix learning but but the principle applies the same so here's an example of a deer slate prior distribution over some categorical parameters so let's say our $\mathbf{a}$ matrix or a array let's just say it's a matrix so now the random variable itself are the entries of the $\mathbf{a}$ matrix a prior of over those entries is something called a deer slay distribution which is just a vector of real positive real numbers and here we've reshaped it so that it has the shape of an area so let's say this array represents like the payoff Matrix in our Bandit task so the two columns are two possible hidden states are context one context two and then the two observations are punishment and reward so if this is our prior which is measured by these deer slay um parameters the likelihood distribution that they parametrize also you can express this as the expected value of the dersland distribution is a categorical with these values so what you can see is that the deer Chalet um counts like the the num the kind of scalar magnitude of the deer slate parameters kind of represent a prior confidence about the probability of um of the different contingencies encoded in the $\mathbf{a}$ matrix so I made these these very simple um here like the the gear slave priors are not nine counts for seeing punishment given context and one count for seeing reward given context one but you can change those to very very high numbers so for instance if that 9 on the left was changed to a ten thousand then on the right the probabilities would become like 0.9999 to .0001 so the scale of the deer slay also known as pseudo count parameters encode something like a belief about how many times that particular Coincidence of observation in Hidden state has been observed which you can also think of as like a prior confidence about that contingency um and the expectation to create your $\mathbf{a}$ matrix via this uh

going from a deer slate to an array is very simple because you just take each deer slate count and divide it by the column wise sum which is this a naught variable on the lower right so it's just that particular value of the the a array and divided by the sum of the counts um so that's how you go from a deer site prior over a categorical likelihood distribution through the expectation um and then inference and I'm not going to get into like the variational posterior but it's very simple so when you're doing learning and you're trying to actually update these deer parameters as a function of observations what essentially you have is a new posterior that's not now not over hidden states of the world but you have a variational posterior over the parameters the categorical or rather a variational posterior over the deer Schley distribution the deer distribution that parameterizes your a matrix so that's what's representing the lower right so let's assume that our beliefs about the deer shlay parameters were as it is now so your beliefs about the a matrix and then we get an observation and let's say that we saw the observation of punishment and that can be represented by this one hot vector now we want to update our beliefs about uh the a matrix given the observation but what to do that what you also need is a hidden state or a belief about hidden States and let's say that the agent uh was very confident that the hidden state was context one what ends up happening to the resulting posterior over the a matrix is a very simple associative quasi-associative learning rule where the update to the a the deer slay represented by this um bold a Sub-Q becomes the prior over the deer slight parameters plus the outer product between the observation vector and the hidden State belief Vector so this is kind of like a form of coincidence detection where you just see what parts of the Hidden States line up with what parts of the observations and you increment your beliefs about the a matrix accordingly so in this case if I really believe the context was one and I saw a punishment then my belief about that particular contingency seeing punishment under context one would just be incremented by a plus one and that's what this outer product at the lower right calculates it computes a matrix that is the increment to your beliefs about the a matrix so in this case The Matrix would be have one in the upper left and then zeros in the other three entries and you use that to kind of increment your your a matrix technically you're incrementing a deer Chalet kind of conjugate prior over your a matrix but it's a very a variational it's really a variational dirichlet posterior but you can imagine if your hidden states are also contaminated with uncertainty so say you had uh 50 your 50 50 whether it was context one or context two then similarly with this a matrix update the update would then be spread over the two um possible contingencies so if you saw an observation but you weren't sure what the hidden state was then both that the just the overall

probability of seeing punishment under your generative model would go up because you would increment that entire row of the A matrix within this case 0.5 so that's just an important thing to note is that uncertainty in your own hidden State beliefs will bleed into the updates to the dirichlet parameters and oftentimes alongside uh I mean it optimally alongside a matrix learning or parameter learning you're also going to be augmenting your expected free energy because your generative model is now different so you have these new priors and posteriors with a so-called parameter Information Gain term or a novelty term so while you're pursuing learning it also makes sense to choose your policies such that you'll maximize um a kind of the information you get based on the consequences of your policies about the parameters of the generative model so this this expected KL Divergence exactly quantifies how much a given policy will lead to a a good Bayesian update of your parameter beliefs so intuitively this quantity might be um described as how much do I expect the consequences of my actions will update my beliefs about parameters so consequences of actions is represented by that Q of O given P_i the thing that the uh KL Divergence is taking taken under expectation of and then the actual KL Divergence is saying how much surprise will I get given these observations um relative to my current beliefs about the the parameters of the generative model so this is what's called the literature of the novelty term and it's very easy to experimentally turn off or on this novelty term using simple Flags in in the agent class and this goes for the other components of the variation or the expected free energy as well you can kind of turn on flags that say do I want to use this novelty or parameter information game do I want to use the state info gain which is the same as the epistemic value or the Bayesian surprise and then do I want to use the utility so for the learning simulations it makes sense if you're doing learning to have to make sure that this parameter is turned on um yeah okay so that's that's the kind of um slides on on learning and now we can actually get into implementing this in primevp um so we first will create an A matrix and we'll let the agent have very slight positive beliefs about the reward contingencies and I'll explain later why this helps it helps with learning if they don't have total ignorance about the reward probabilities but they have some bias in some direction it doesn't actually have to be 0.51 it could be the other way um and then there's utility functions that allow you to create dearishly variables that have the same shape as some base array so here we this p_a variable is basically the prior that bold a sub P variable that represents the agent's dirichlet prior over the A matrix so this is now a new variable that we're going to pass in to the um to the agent Constructor so if our A matrix is like let's look at the um you know the reward contingency reward modality so that's A_1 so this is the agent's current beliefs about the the

payoff structure right so under choose left and choose right those are the two contingencies given the two hidden States PA will have the same except that it'll have those contingencies encoded in terms of dear slay pseudo counts so if I made them made this pseudo count scale 10. then they are very con or quasi like relatively confident that the payoff probabilities look like but again these are not probabilities they're dear slay parameters that parameterize a um categorical likelihood so you can use the normalized distribution function to actually then visualize the dirichlet distribution in terms of actual categorical parameters which so taking the expectation I.E the normalization of the dear slate prior will give you exactly this so if I say are these two things the same they should be the same okay maybe down to numerical differences they're not but yeah um okay so yeah that's how you parameterize that so we'll create a this is an important Point that's also used in SPM often we want certain contingencies to be unavailable to learning so we assume these are contingency that are baked into the agent's beliefs about the world and that's not adaptable so one way and this is just borrowed from the SPM way of doing it to encode that is to bake in a kind of um really precise confidence or very high confidence that certain contingencies are the case and you can just do that by adjusting the scale parameter for particular indices so for instance by doing this I'm just telling the agent that it it essentially doesn't learn the contingencies related to the null so for instance if if you're in the uh start state it believes with really high confidence that you'll always get the null observation and we just operationalize that by creating very very high pseudo count or dear slay priors over that particular slice of the a matrix um so that's an important point to to do and then we'll just write all this stuff so this prior account about the null observations and then the scale that determines their General confidence outside of this this um null thing we'll write that all into a function so that we can then parameterize our PA and then we now write a new active inference with learning Loop where instead of just uh storing the history of choices and beliefs we also store the history of their beliefs about the dirichlet parameters um and an important thing that you can also do in primevp which is nice is you can say I only want to learn particular modalities so when you're creating the agent class there's a bunch of arguments you can pass in but one of them is I only want to be learning the reward payoff modality right now and this can change you could have them learn the the hint accuracy as well but by passing in a list of which of the modality indices that you want them to learn you turn off learning on all the other modalities which normally in SPM more or what we used to have to do in pi mdp is you just have to turn up those dear pseudo counts super high on all the modalities that you don't want to learn but um if there are other

modalities that you don't want to learn now you just you can just pass in this list of modalities that you want to learn and it'll only focus learning onto that so now I'm setting up an agent as these particular beliefs about the world via a and now has a prior that it's going to update through doing variational inference or learning and then so we'll create the generative model here including this new prior over a and then we'll run the active inference loop with learning which involves creating an agent that can only do learning on the payoff structure modality so we want the agent to now learn which arm is best basically because it doesn't know the payoffs and then also to use parameter Information Gain to motivate its decision and there's another thing I forgot to mention which is the learning rate which is how much they increment their beliefs about the or about the a matrix using this update row so there's something I didn't mention there's often a learning rate that's that's added here that basically scales how big this update is so that can be you know you can experiment it depends on the on the application how what what the scale of that is going to be but the default is one so in this in this example the agent gets the hint for one time step he's got the standard like negative four and then it goes straight to the right because it believes the um the hint is accurate so it gets a reward is right then it goes and starts playing the right arm and uh you see it's posterior beliefs instantly go to right and now so since what we've also spit out of this active infant soup is the beliefs about the um a matrix in this QA hist posterior over a history we can plot its beliefs about the a um the a matrix probability over the contingency of seeing reward given that I was in the right arm and what you see is that even though they start with basically 50 50 beliefs as they gather observations their beliefs about that posterior probability over that particular entry of the a matrix get bigger and bigger an interesting consequence of this is their beliefs about the left arm contingencies don't change because they don't ever experience that that um state of the world so that's like an interesting um what you might call in uh in machine learning like a it's not really a bad bootstrap but it's the idea of um like selective sampling so you only learn those contingencies that are selective to the part of the world that you're sampling so I don't the agent doesn't know what that other slice of the a matrix looks like because it's never experienced what it's like to be in the left is better context and and playing the left arm so it has no it's posterior beliefs about that part of the a matrix remain the same but you can see that over time its beliefs start to converge to point a which is what we set in the uh in the generative model this is just basic statistical learning right it's just getting reward observations over time and as it gets a sequence of reward and Punishment through this very associative basic mechanism is just incrementing its dear slay beliefs and what I'm showing is the

normalized IED expectation of those beliefs over time so you get something that's 0.8 but the actual dirichlet parameters themselves will be growing um like linearly in time effectively depending on the observations they get um yeah so that is just a basic learning and then there's another thing you can do which is take advantage of this change probability so now we allow the context to change and this is one way to get them to actually be able to explore and learn more about both reward probabilities in the landscape is allow them to have not only to make the environment change but they also entertain beliefs in their in their uh B Matrix that the the environment can change so you could use this to do B Matrix learning as well by defining like a PB variable but here we're just going to have a matrix learning as we were doing before I've changed a little bit the parameters to make things like a little more stable um because sometimes you get weird Behavior I mean that's something we can play around with too so now they allow the the environment itself can change and they also in their B Matrix they think the environment can change and then I'll plot uh the history of beliefs and choices so now they instantly go for um playing the arms they kind of risk it they don't the parameter Information Gain actually outweighs the epistemic or state information game they start playing they gather observations but they're not very confident about what the state of the world is because they only are getting information about the hidden state from the actual sequence of rewards and losses and then eventually they go to the hint because probably the utility becomes low because they don't know what the reward probabilities are and they're building their own beliefs then they get the hint then the hint is accurate so they have very precise beliefs about the hidden and then the the world actually switches so you see the Red Dot is the actual switching statistics of the Bandit and now they've kind of explored wolf band at Arms while also updating their the bandits and you see that their beliefs about their left arm are kind of bad because they they it seems like they probably mostly gotten negative data while they're in the left arm and they didn't explore it very much um and and yeah or no I guess at the end they're exploring it quite a bit but their their beliefs about it go down because they have I guess a bad experience with it um yeah that's a left arm that's the right arm but you can like keep trying this and mess around with different parameters and you'll get different sorts of uh beliefs okay so here in this example there's some switching going on the posters aren't tracking very well until they actually visit the hint and then their posters start getting much more um much more precise and then at that point they can start making more precise beliefs in this case they ended up kind of doing poorly on learning on both of them because the true the word probabilities are PA yeah um but so everything I've talked about so far is an

example where learning can often lead to sub-optimal outcomes because it's done in an online fashion so this is an interesting Point of Departure between SPM and pine DP but I mean you can do all the same things in SPM that you can do in pi VP so here what I'm doing is I'm updating the a matrix every time I get an observation so they do hidden State inference they select policies they select actions and then given the last observation and the last hidden State belief they do one step of parameter inference and they use the next a matrix that's what's handled in this update a function of of the agent class they update their beliefs about the a matrix and they use that for inference at the next time step another way you could do this is you could do an entire trial and then update your a matrix at the very end of all the time steps of action selection and observation sampling and then use that a matrix for another trial so this is kind of the difference between um like a more em uh like a separation of time scales em or expectation maximization approach where given a set of observations then I update my model and then I go back and I do more trials without updating my model but using the same fixed one and then you can kind of do that in epochs and that's typically how it's done in SPM because you only do parameter learning basically this part outside this time Loop so you do it like down here but then what you do is you run multiple trials and the the nice thing about that is it kind of makes the a matrix updates less um likely to in the moment bias your action selection so for a given set of time steps you're you're locked into what your a matrix beliefs are and you don't do the update until a set of time steps has elapsed and uh yeah so for the purposes of this tutorial I just did this online learning thing but as you can see that can lead to actually weirdly sub-optimal Behavior where they don't learn the true reward statistics very well and it depends on a bunch of things too like the learning rate and you know there's a I haven't explored this demo as much as I would like but there's a lot of interesting kind of side effects like here they're actually learning very weird statistics about the the Bandit landscape like they start kind of learning it well but then as their observations keep going they don't they don't actually learn very well here they just avoid playing because they have a bad early experience so this is an example of like a bad bootstrap and then they go and just play the hint because they're too scared of anything else even though they're hidden State inference is because the hint perfectly tracks the hidden state so yeah there's it's just um this is just a kind of meant to be an example of what you can do with learning um but there's there's going to be examples where learning actually causes sub-optimal Behavior but in an interesting way so here's an example where they actually kind of are learning the correct reward statistics things are starting to converge to point eight for both Bandit arms but um but it's very stochastic as

you can see and I also chose action selection to be stochastic with deterministic I wonder what you would get um yeah yeah they're just Arc maxing the expected free energy or Arc meaning so hint is always the most safest option given their history of uh if I can make one remark on that kind of diversity behaviors that we're seeing pi mdp and active inference are providing us a space and a approach to composable generative model construction that we can then sift through and to co-evolve with to find different strategies and behaviors so active inference or even any specified generative model it's not an answer or a solution for example to the explore exploit dilemma we see this like empirically right now active inference is not in general resolving explore exploit anygm in general is not resolving explore exploit any parameterization is not resolving explore exploit even for one environment it just is equivalent to saying like well we have a linear model of healthcare data so we've resolved the health care issue and um this really this really shows the space that we build in and what remains to be built is so open yeah absolutely yeah I mean because optimality is not really a function of what the generative model is or what the algorithm is what's it's more guaranteed to be optimal based on how analogous the generative model and the generative process are so if the generative model is a perfect model of the generative process then doing Bayesian inference with respect to that generative model that is optimal for like everything but um but uh how you learn the generative model that's what we're seeing here if you learn the generative model in online fashion it's not guaranteed to learn the right to generative model in many cases it learns the wrong one and this is something that a lot of people in active inference world have explored for instance nor Sajid who's also a student of Carl's has explored a lot the the limitations and the the boundaries of parameter learning and things like preference learning like what if you were learning the C Vector itself how can that be learned online in an Adaptive way um yeah and also Alec chance has that paper action oriented models where they're learning the B Matrix and in that thing they're showing how the agent learns kind of sub-optimal strategies depending on its exploration of parameter space um but that also actually is kind of a one of the benefits of active inference that paper shows because it shows if you have an agent that does have epistemic value the model it learns of the world is like better than agents that don't have epistemic value but again that the models are always action oriented they're always based on what parts of the full State space is the agent driven to sample awesome Jacob any closing comments and then all the closing comment and then Connor with the last word yeah well thank you very much for the great um overview what particularly uh was inter um I I found particular particularly interesting was um both how you

touched on how it could be modified uh with the action perception look could be modified with the other other modules even though uh as you mentioned there would be more opportunities to go deeper in that but also on the on learning um the the duration parameters and I would be interested to um see how that uh also influences planning uh like if you would up if you would do like um kind of pseudo update of your duration parameters when you're calculating your expected free energy as in what would I uh I'm calculating my expected free whilst also taking into account that my B would change on each in on each step and uh how that how that changes like the optimality of the temporal depth that we do planning with I think it's super super interesting and really looking forward to hopefully uh another model stream yeah that's a um okay is it okay if I quickly respond yeah please please um yeah that that's a very interesting point that you just made about um these this fictive or imagined update whereas that I had a yeah this slide so that term that novelty term in theory that does capture exactly what you're saying yakub is that this thing says how would my dear straight parameters update if I was to take this policy pie and then I use that term to actually choose where to explore next so if I turn on the parameter information game term in the agent class they'll much more quickly skip that skip the hint and go directly to sampling the bandits because they're driven by novelty oh I want to know what the reward probabilities are so I'm going to go explore however when it doesn't do and what I think you're intimating with your comment is that if I did like multi-step planning two time steps in the future will I be able to do my planning given how I think I updated my parameters at the first time step so that's what the equivalent of sophisticated inference sophisticated inference is saying how would I plan at time step three given how I think my beliefs would update up to time step two and that's done right now for hidden States but I haven't seen that done for parameters and I think there's someone like a student of Ryan Smith's uh is working on that now um is like this propagation of counterfactual beliefs about how my parameter beliefs would change in the future um and that's like that's like a really cutting edge I think active inference it's like not only novelty but how would my like prospective sophistication about how your own parameter beliefs will have evolved by time Point T and then using that to do planning for time Point t plus one um that's like a really sophisticated stuff I think getting just even Bare Bones sophisticated inference with hidden State counterfactual stuff in in Pine BP that would be a huge accomplishment too and then of course incorporating it to be more um more uh with the parameter sophistication as well and then another quick thing is I can quickly show you um an example of one of those other oopsie just while you're finding it the live that that last

discussion on uh sophisticated planning it's something that at the semantic level we engage with every day what courses should I take this quarter so that I can learn what I don't know today so that next quarter I'll be able to make a better plan for which classes to take so that in three years when I graduate but it's already phrased at the semantic granularity that these models are rapidly converging towards and they're not converging towards it by scaling is all you need they're converging towards it with a factorized actually semantic approach which is very exciting yeah that's a great analogy it's like by the if you were a naive active inference agent without sophistication you would never plan with like a 10 time step planning Horizon you never plan to take or let's say a three semester plan Rising you never plan to take multivariate calculus in your third semester because you will you would not have anticipated by the second semester I now have enough linear algebra to take I know that by the time I finish the second semester I will now know linear algebra so I'll be well suited to take multivariable calculus whereas what we as humans do is we do have that parameter sophistication I can plan as a freshman as a first year to take in my third or fourth semester some high Advanced physics because I know by that time I will have the requisite multivariable calculus or whatever so that's a that's a really nice analogy that I never I think that's a good uh a good example welcome to active View yeah exactly um so here uh I just wanted to show you Jacob um here's an example of a hierarchy hierarchical active inference demo I can share this too it's another collab notebook where we're exactly doing we're composing to palmdps so there's like a high level Palm DP and then there's a low level Palm DP and for example when you're doing a step of updating the empirical priors at the high level before you pass them down to be empirical priors at the low level we use the control module for example to do get expected States or use the high level beliefs the high level B Matrix and then the high level chosen action to propagate forward the next posterior beliefs at the high level and then those things themselves parameterize a low level empirical prior for this faster Palm DP that's going on at the low level so this is an example where you're composing like agent class calls like this but you're composing them with functions that are built from sub modules and adjacent modules of prime DP um so that's an example of the thing you were talking about but yeah that's very brief like we can get into that um later on in a different stream or something like that yeah awesome Yes uh dot three whenever the time is right I'll I'll just give two closing areas again really appreciate coming back on and sharing this development in progress already it feels more powerful and documented than when we were in the dot one just a few weeks or months ago I think two areas that are going to be really exciting to discuss and see how they're

implemented and also the plurality of ways that they're implemented the first area is structured learning on cognitive models from the outside so as a ethologist as a behavioral researcher how do we do structure learning on cognitive models four systems that we actually know about their cognitive architecture or not but also the view from the inside in terms of structured learning and metacognition like how should I change the dimensionality of my B Matrix or should I turn on that flag to engage in this kind of sophisticated inference so structure learning on cognitive models view from the outside View From the Inside is one exciting area and the second more experimental area is statistical power analysis pre and post-hoc statistical power analysis along the lines of the design Matrix in SPM so that you put 15 time steps and 0.8.2 and punishment is for and then you could sweep across parameters and learn about how well it did but if there was certain interfaces analytical or numerical approaches to be like yes with 25 time steps we have this much of an expectation of convergence or this differential in rewards should be resolvable by an Adaptive agent over this long and just understand how long should these experiments be because there are such interesting results with One-Shot learning and with being able to generate someone's voice from just 10 seconds of them talking or make a video of somebody as a deep fake with just a still image and so it seems like it's possible to learn a lot from a little and if we can learn a lot from a little and have a semantic cognitive model that would be quite great absolutely yeah like it's almost like hyper parameter optimization on the landscape of active inference models like how do I choose the parameters of an active inference model in a smart way um yeah there's several methods for doing that like yeah that we could uh definitely explore yeah data efficiency is the main main thing like you said like making it so you don't have to train it on a trillion images like with a deep neural network perfect and Dot 007.2 and whenever you want to join you in any colleagues are are always welcome to share the next spiral in pi mdp development awesome thank you so much again for letting me come on and listening I hope it was helpful um yeah and I'm glad that it's recorded it's really an amazing resource that you guys are developing here so thanks again thank you till next time bye